# Retina project - SNN

## Group 13

Antonino Davolos, Sandra Sandström, Felix Hessinger, Aamer Barood, Hind Dafallah, Christos Michail

March 25, 2025

LULEÅ TEKNISKA UNIVERSITET

# Group Presentation - Team Retina 13

(a) Sandra

(b) Felix

(c) Hind

(d) Christos

(e) Antonino

(f) Aamer

# Table of Contents

1. Introduction
   - Goal
   - Our Dataset
2. Method
   - Preprocessing
   - SW library employed
   - Network Architecture
3. Results
4. Discussion & Future work

# Project Goal

- Develop a vision system using **Spiking Neural Networks (SNNs)**.
- Detect specific **visual motion patterns** (e.g., leftward, rightward, upward or downward motion) of a given object of interest within a sequence of frames.
- Create an SNN that responds to **directional movements**.
- Utilize **simulated input receptors** or **real event-based data** from neuromorphic vision sensors.
- Leverage the temporal dynamics of SNNs for efficient and biologically inspired motion detection.

LULEÅ TEKNISKA UNIVERSITET

# VOT2015 Dataset

**Dataset Overview:**

- From Visual Object Tracking (VOT) Challenge 2015
- 60 short sequences (avg. ∼370 frames), multiple frame sizes, at 30 fps
- Fully annotated with rotated bounding boxes
- Publicly available research dataset for fair comparison with SOTA methods

**Ground Truth:**

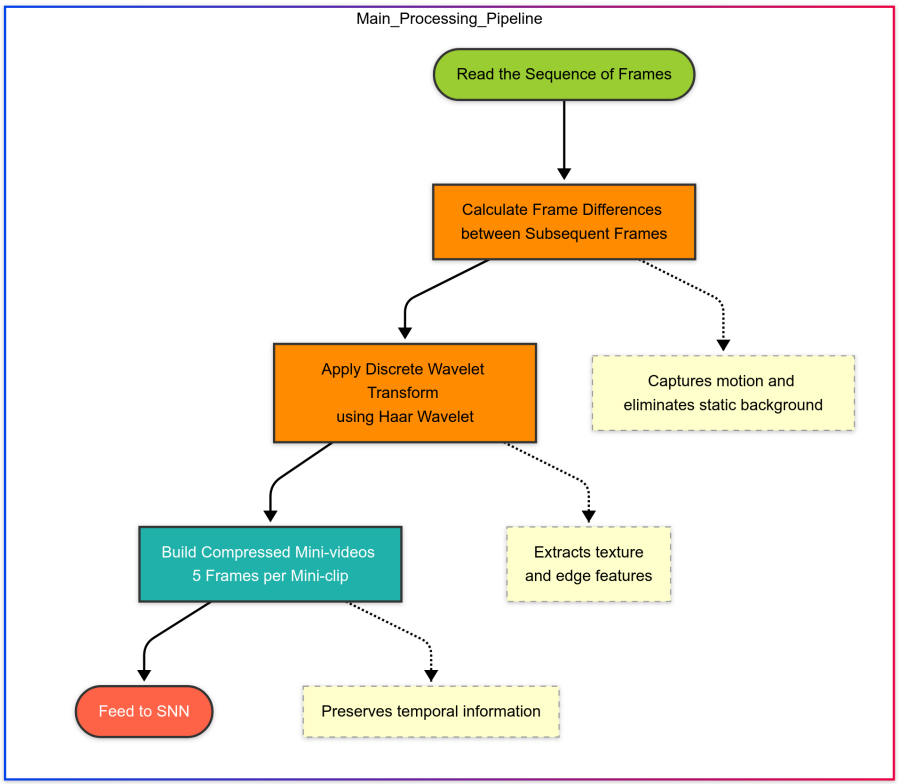
- Frame-by-frame manual annotation
- Rotated bounding boxes for precision
- Ground Truth Encoding: Avg of 4 vertices

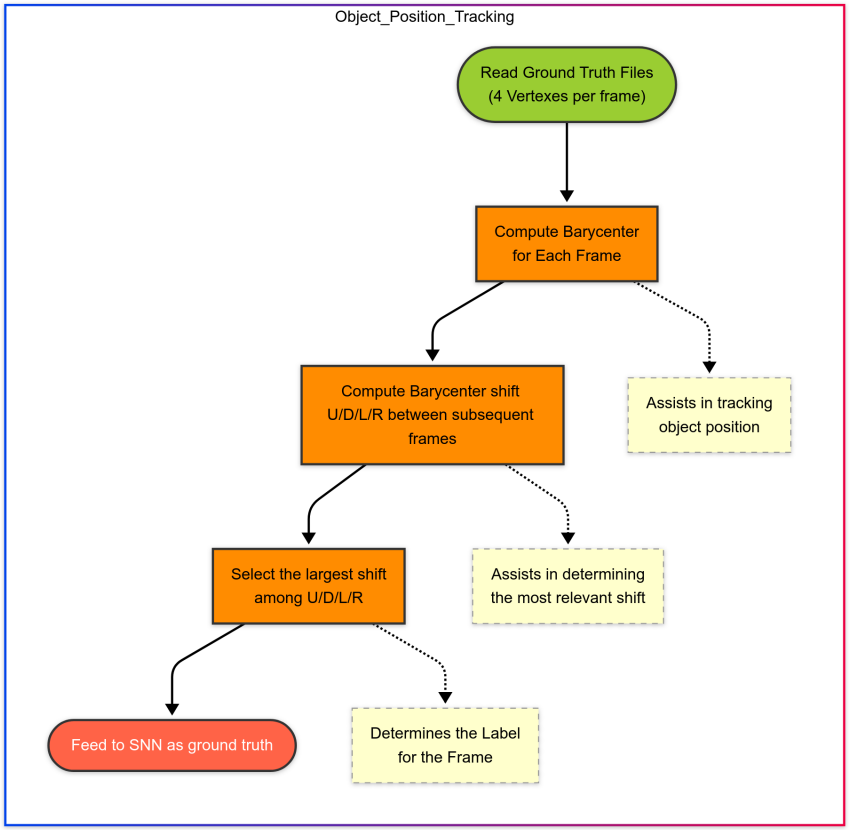
**Challenging Aspects:**

- Illumination changes
- Occlusion
- Motion blur
- Size changes
- Camera motion

LULEÅ TEKNISKA UNIVERSITET

# Data Preprocessing




- Each sequence split into 70% (train-), 15% (validation-), 15% (test-) sets.
- Each sequence trained separately with same hyperparameters.

# Brian2: Overview

- Standalone Python-based simulator for fast numerical simulations.
- Flexible syntax for defining complex neuron models.
- Efficient event-based simulation with custom differential equations.
- Ideal for studying biological neuron dynamics and computational neuroscience.
- Focuses on scientific modeling of biological neurons and networks.

LULEÅ TEKNISKA UNIVERSITET

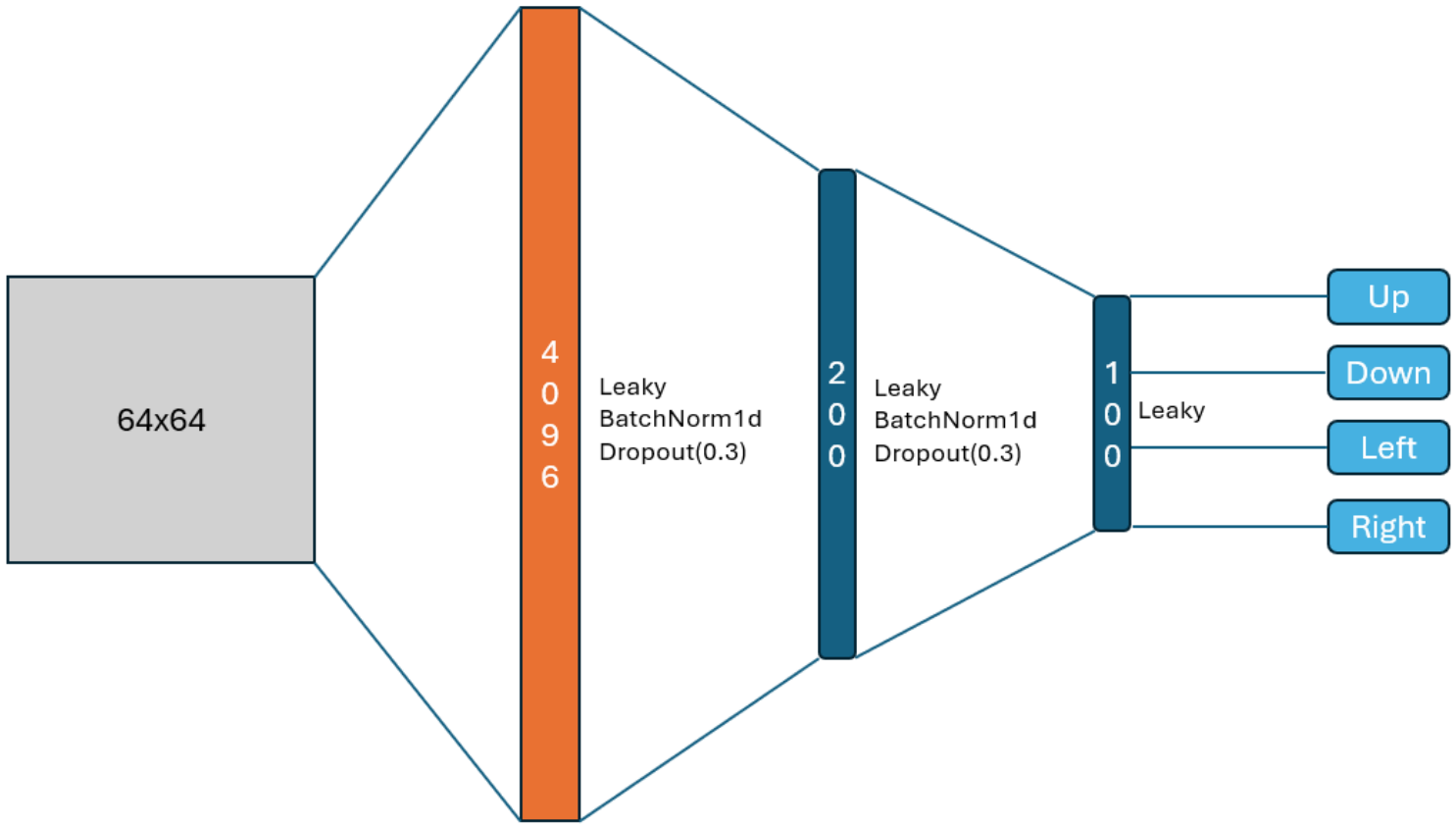
# SNN Torch: Overview

- Built on PyTorch, optimized for time-varying data.
- Enables easy integration of SNNs with deep learning models.
- Designed for neuromorphic computing and deep SNNs.
- Focuses on machine learning, real-time processing, and optimization.

**Project Choice:** Our project relies on time-varying data in deep learning and neuromorphic computing, so we chose `snnTorch`.

LULEÅ TEKNISKA UNIVERSITET

# SNN Architecture - Visualisation




LUI TEKNISK UNIVERSITET

# SNN Architecture - Code

```
class SNNModel(nn.Module):
    def __init__(self, input_size=4096, hidden_size=200, output_size=4, beta=0.6):
        super().__init__()
        self.fc1 = nn.Linear(input_size, hidden_size)
        self.lif1 = snn.Leaky(beta=beta)
        self.bn1 = nn.BatchNorm1d(hidden_size)
        self.dropout1 = nn.Dropout(0.3)

        self.fc2 = nn.Linear(hidden_size, hidden_size // 2)
        self.lif2 = snn.Leaky(beta=beta)
        self.bn2 = nn.BatchNorm1d(hidden_size // 2)
        self.dropout2 = nn.Dropout(0.3)

        self.fc3 = nn.Linear(hidden_size // 2, output_size)
        self.lif3 = snn.Leaky(beta=beta)
```

- **nn.Linear(input_size, output_size)**:
  Defines the input and output size of the layer.
- **snn.Leaky(beta)**:
  spike declination speed, affects memory. Lower = shorter memory.
  We chose **0.69** - moderate.
- **nn.BatchNorm1d(layer_size)**:
  Normalizes activations across batch. Stabilizes training and speeds up convergence.
- **nn.Dropout(0.3)**:
  Randomly drops 30% of neurons to prevent overfitting.

LULEÅ TEKNISKA UNIVERSITET

# Hyperparameters for SNN-Based Movement Tracking

Hyperarameters determined by trial and error on validation set statistics

- $\beta$: 0.69 [hardest param to tune: different movement speeds benefit from different $\beta$'s]
- **Initial Learning Rate (LR)**: 0.07
- **Adaptive Learning Rate (aLR) Strategy**: `ReduceLROnPlateau`
  - Factor: 0.75 (LR reduced by 25% when validation loss plateaus)
  - Patience: 15 epochs (Waits before reducing LR)
  - Minimum LR: $10^{-6}$
- **Early Stopping**:
  - Patience: 100 epochs (Stops if no improvement)
  - Best model (on validation loss) is saved automatically
- **Number of Epochs**: 500
- **Optimizer**: Stochastic Gradient Descent (SGD)
- **Loss Function**: Cross-Entropy Loss

# Forward pass

```python
def forward(self, x):
    # Initialize membrane potentials
    mem1 = self.lif1.init_leaky()
    mem2 = self.lif2.init_leaky()
    mem3 = self.lif3.init_leaky()

    batch_size = x.shape[0]

    for t in range(5):

      frame_batch = x[:, t, :, :]  # Shape: [batch_size, frame_no, H, W]

      flattened_tensor = frame_batch.view(batch_size, -1)

      # First hidden layer
      cur1 = self.fc1(flattened_tensor)
      spk1, mem1 = self.lif1(cur1, mem1)
      spk1 = self.bn1(spk1)
      spk1 = self.dropout1(spk1)

      # Second hidden layer
      cur2 = self.fc2(spk1)
      spk2, mem2 = self.lif2(cur2, mem2)
      spk2 = self.bn2(spk2)
      spk2 = self.dropout2(spk2)

      # Output layer
      cur3 = self.fc3(spk2)
      spk3, mem3 = self.lif3(cur3, mem3)
    return spk3, [mem3, spk3]
```

LULEÅ TEKNISKA UNIVERSITET

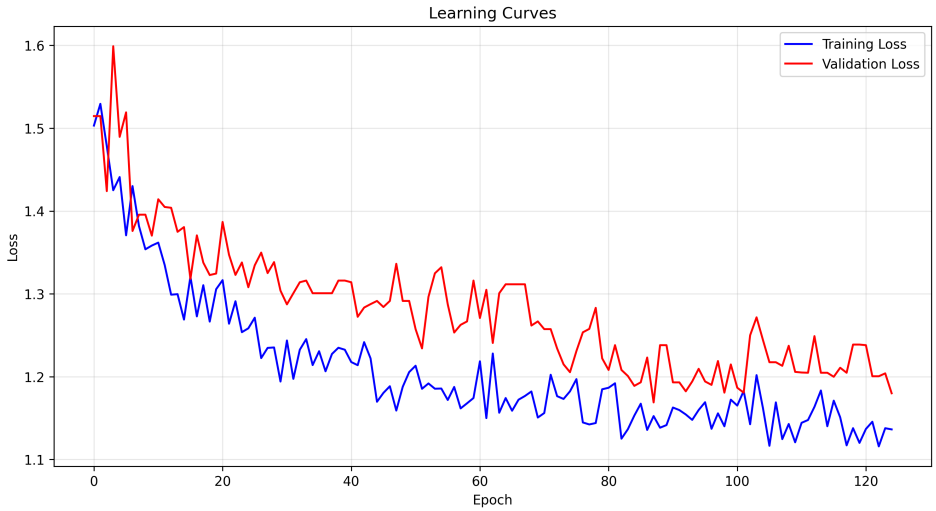
# Click to play

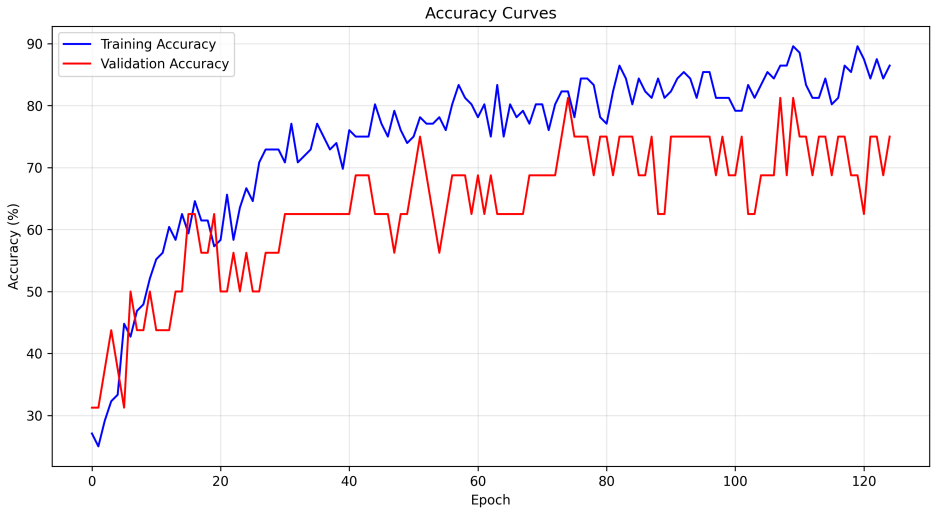
Label distribution:

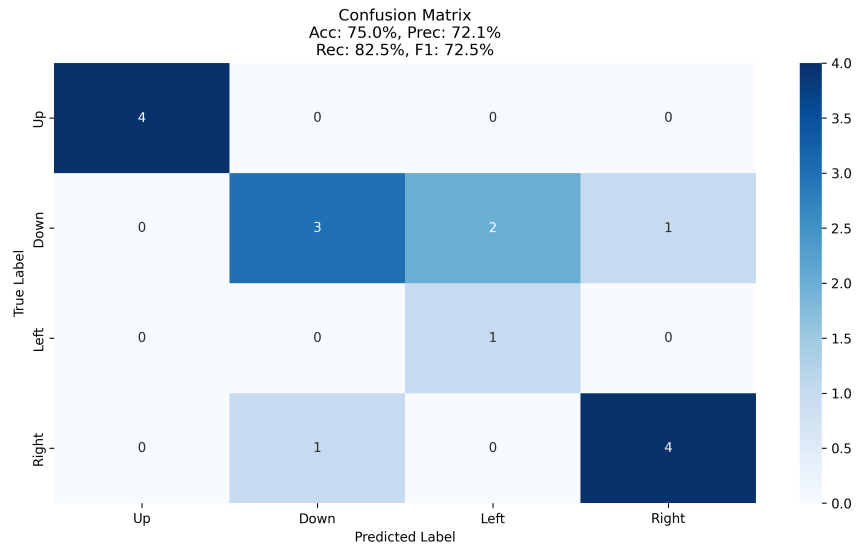
Up: 39 (26.9%) Down: 52 (35.9%) Left: 11 (7.6%) Right: 43 (29.7%)

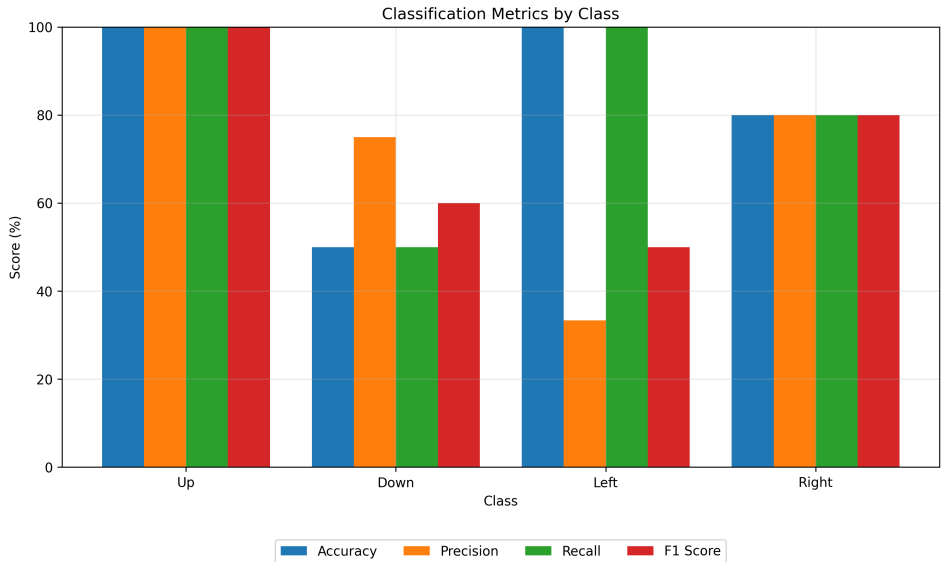
LULEÅ TEKNISKA UNIVERSITET

# Butterfly video results



# More challenging videos

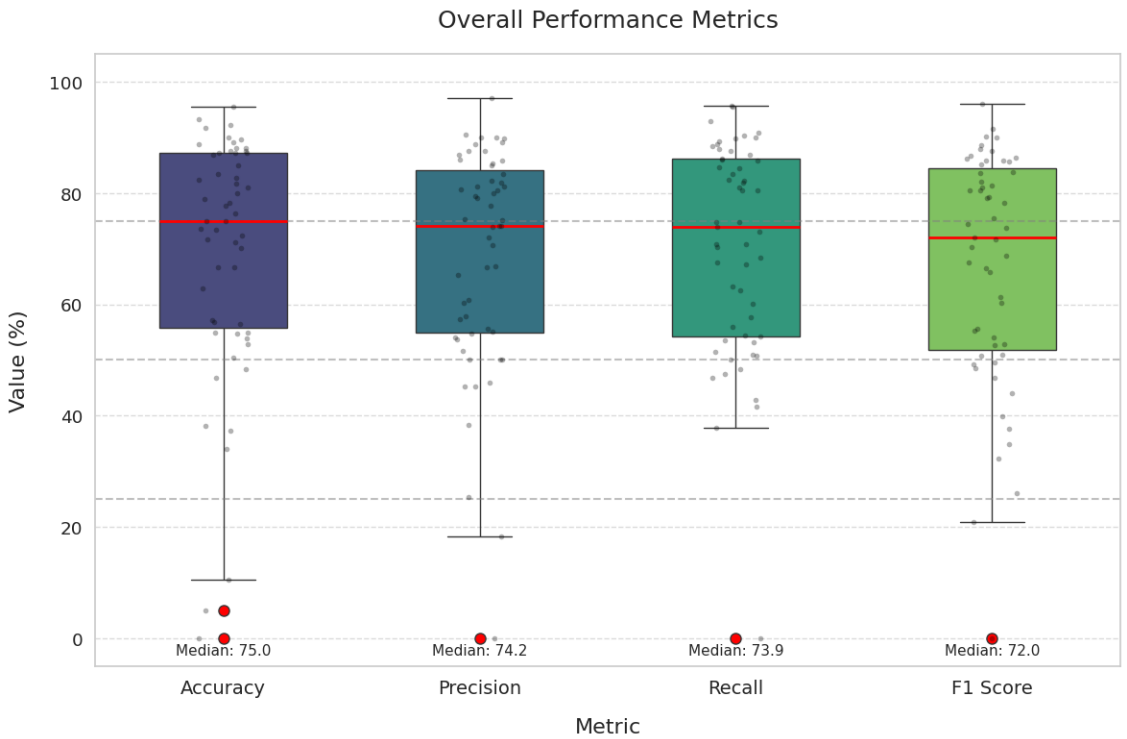
Click to play

LULEÅ TEKNISKA UNIVERSITET

# Overall Results

Average of the Confusion Matrix metrics across 55 frame sequences:

| Metric | Mean ± Std | Min | Max |
|---|---|---|---|
| Accuracy | 69.22 ± 22.24 | 0.00 | 95.45 |
| Overall Precision | 68.54 ± 20.08 | 0.00 | 97.06 |
| Overall Recall | 70.31 ± 19.12 | 0.00 | 95.64 |
| Overall F1 Score | 66.76 ± 21.08 | 0.00 | 95.94 |




LULEÅ TEKNISKA UNIVERSITET

# Classification metrics by Motion Direction

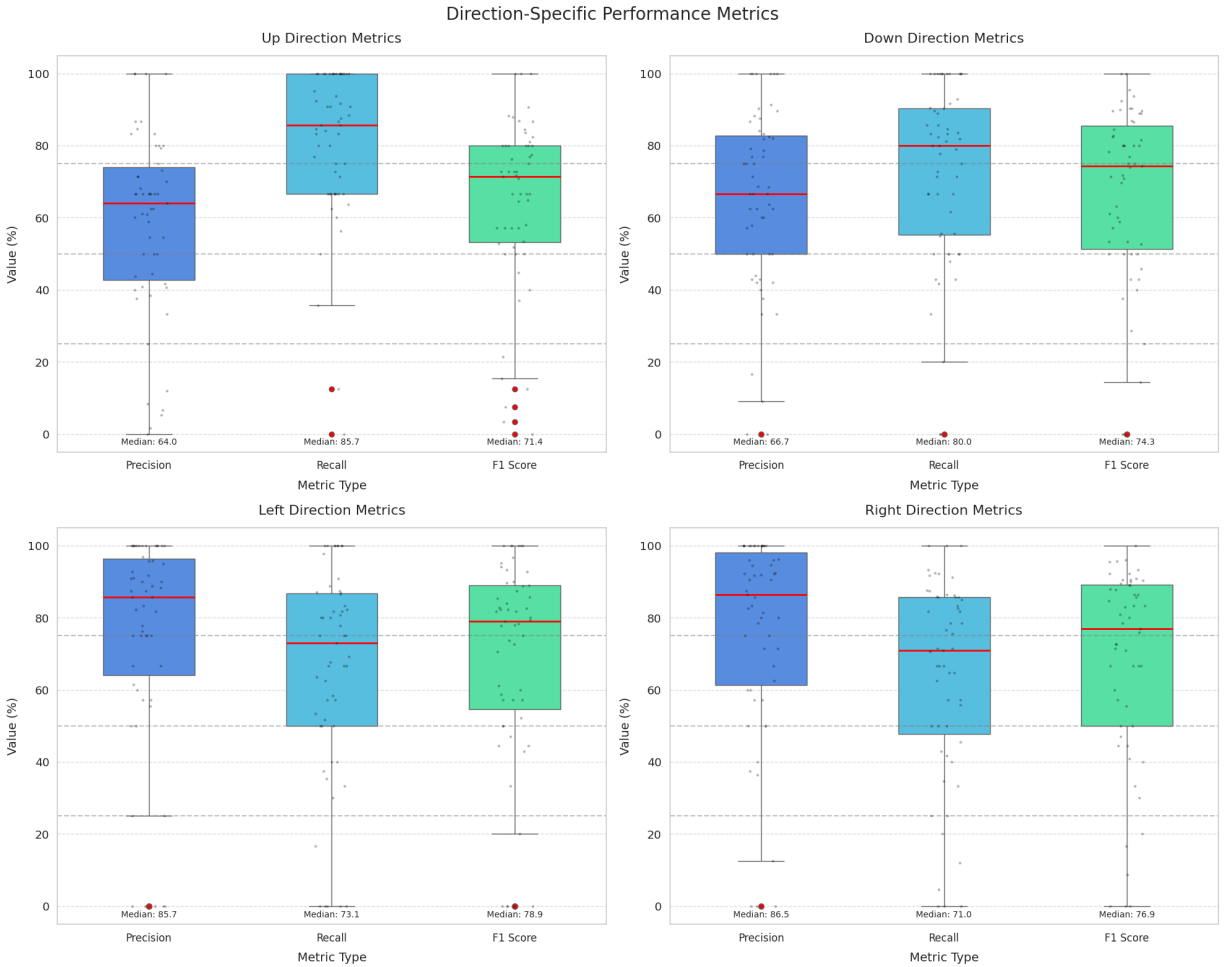



Figure: Box plot representation by class

# Empirical Observations on the dataset

## Video Length (frames)

- Length avg[std]: $370.67 \pm 271.45$ frames
- shortest: 41 frames (0% on all metrics)
- longest: 1500 frames (88% F1 score & 88% accuracy)

## Class Imbalance

- Some sequences have little to none UP/DOWN or LEFT/RIGHT motion
- Other have more equally distributed classes

LULEÅ TEKNISKA UNIVERSITET

# Let's look at how imbalanced classes are

We computed these two metrics for all 55 video sequences:

## Normalized Std Deviation (NSD)

**Definitions:**

- Total frames: $N$
- Counts for each direction: $U$, $D$, $L$, $R$
- Ideal count per direction: $\mu = \frac{N}{4}$

**NSD Formula:**

$$\text{NSD} = \frac{\sqrt{\frac{(U-\mu)^2+(D-\mu)^2+(L-\mu)^2+(R-\mu)^2}{4}}}{\mu}$$

**Interpretation:**

- NSD = 0: Perfectly balanced dataset.
- Higher NSD: Greater imbalance.

## Entropy

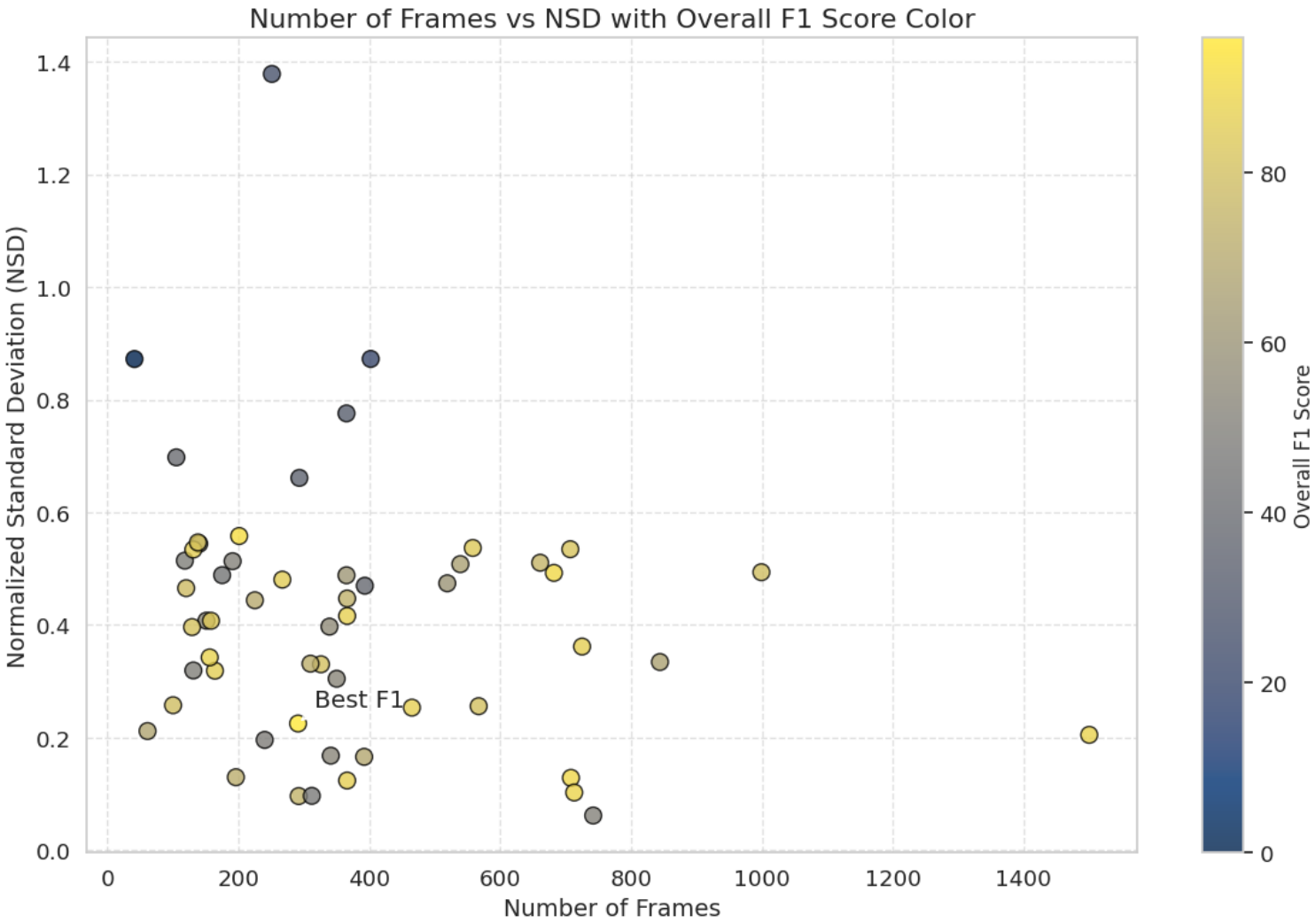
$$H = -\sum_{i \in \{U,D,L,R\}} p_i \log_2 p_i$$

where $p_i = \frac{\text{Count of direction } i}{N}$.

**Key Insight:**

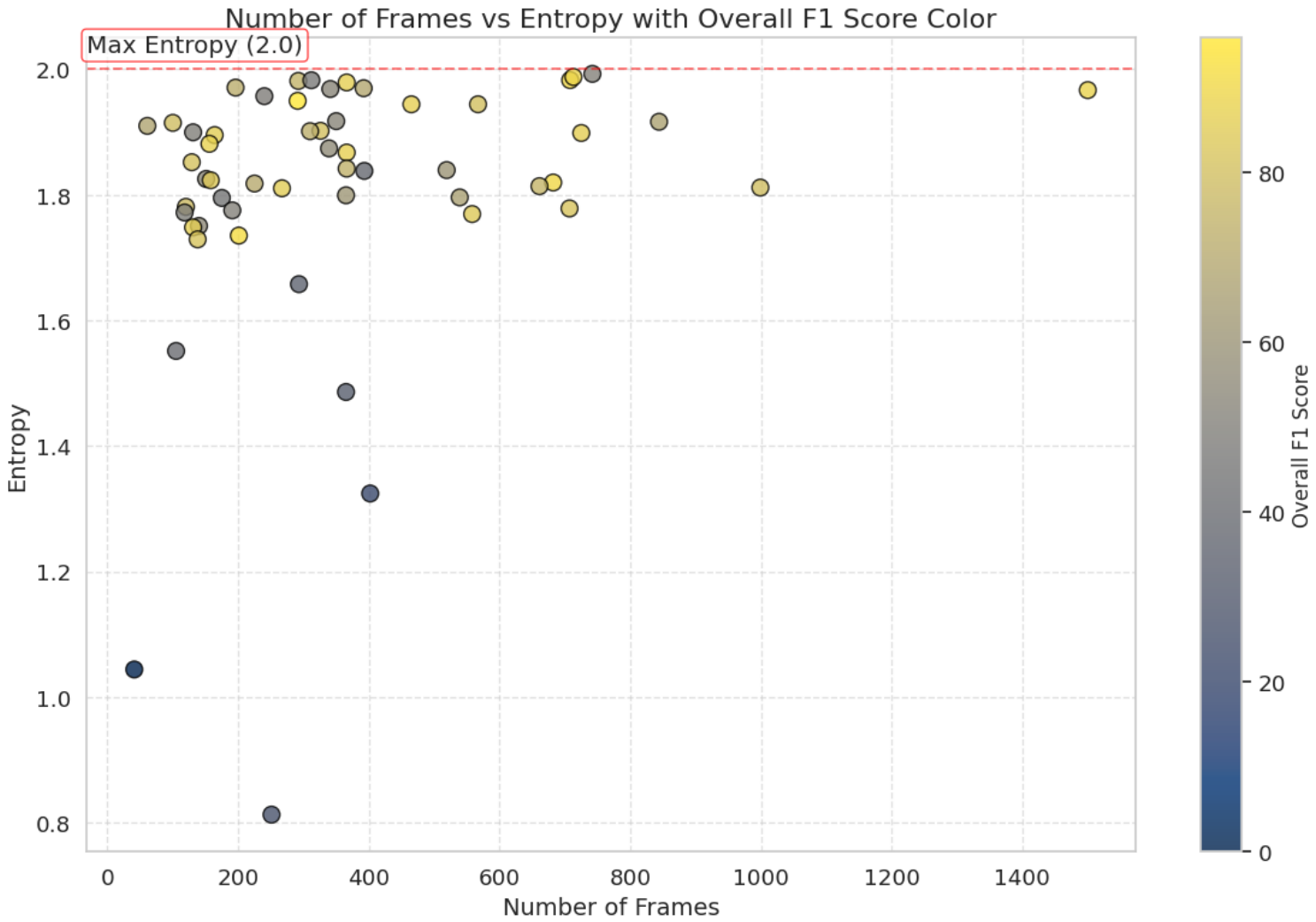
- Entropy reaches its maximum (2) when all directions are equally represented.
- Lower entropy indicates imbalance.

# Imbalance Measure: NSD




LULE TEKNISKA UNIVERSITET

# Imbalance Measure: Entropy




LULE TEKNISKA UNIVERSITET

# Discussion & Future work

**Challenges of Spiking Neural Networks (SNNs)**:

- **Non-differentiability:** The binary nature of spikes (1 for firing, 0 otherwise) makes gradient-based optimization challenging, as gradients cannot flow easily through spiking activation functions
- **Vanishing or Exploding Gradients:** Deep SNNs often face issues with vanishing or exploding gradients during training, particularly when using time-to-first-spike coding in deeper architectures
- **Lack of Robust Training Algorithms:** Current training methods rely on approximations or conversions from conventional neural networks, which are inefficient and fail to fully exploit the unique properties of spiking neurons

LULEÅ TEKNISKA UNIVERSITET

# Discussion & Future work

**Future Work for Spiking Neural Networks (SNNs):**

- Develop **efficient gradient-based training methods** to address the non-differentiability of spike generation, enabling smoother optimization processes
- Implement SNNs on **neuromorphic chips** like Loihi and TrueNorth to maximize energy efficiency and low-latency processing
- Explore biomedical applications, including **brain-machine interfaces** and neural prosthetics for seamless integration with biological systems

LULEÅ TEKNISKA UNIVERSITET